INFORME TÉCNICO FINAL: SISTEMA INTELIGENTE DE DETECCIÓN DE SPAM (SPAMBASE)

Grupo 06 - Tecnologías Emergentes (ISO46B)

Octubre 2026 – Versión 1.0

Institución: Universidad Nacional del Centro del Perú – Facultad de Ingeniería de Sistemas
Equipo: Grupo 06

Resumen Ejecutivo / Abstract

En el presente trabajo se diseñó, implementó y evaluó un sistema integral de clasificación e inferencia en tiempo real para detectar correos electrónicos no deseados (Spam). Partiendo del dataset público UCI Spambase (4,601 registros y 57 características cuantitativas), se estructuró un pipeline libre de fuga de datos (*Data Leakage*), comparando modelos lineales y Redes Neuronales Multicapa (MLP). El modelo promovido a producción (MLP v1) alcanzó un F1-Score de 0.9224 en el conjunto de prueba aislado. La solución se desplegó mediante una API RESTful en FastAPI (`/predecir`, `/salud`) y un Asistente Conversacional en Streamlit con traducción multilingüe al vuelo, explicabilidad de señales y estricto cuidado ético (procesamiento volátil en RAM). La calidad del software fue validada al 100 % mediante una suite de pruebas automáticas con `pytest`.

1. Planteamiento del Problema y Ética (Etapa E1)

1.1. 1.1 Problema y Usuario Objetivo

Las bandejas de entrada se encuentran fuertemente saturadas por mensajes promocionales y estafas (Spam), generando pérdida de tiempo y vulnerabilidad a estafas de *phishing*. El usuario objetivo (estudiantes, profesionales y usuarios comunes) corre el riesgo de traspapelar notificaciones críticas (ofertas laborales, facturas, requerimientos legales) al confundirlas con correo basura.

1.2. 1.2 Momento de Predicción y Datos Disponibles

La predicción ocurre en tiempo real en la interfaz del chatbot, en el preciso instante en que el usuario copia y pega el texto de un correo sospechoso, antes de interactuar con él (hacer clic en un enlace o descargar adjuntos). En ese momento, el sistema extrae un vector de 57 características estadísticas estandarizadas derivado del texto (frecuencias relativas de palabras clave, caracteres $, !, enlaces y secuencias de mayúsculas).

1.3. 1.3 Costo de los Errores y Criterio de Selección de Métrica

- **Falso Positivo (FP):** Clasificar por error un correo legítimo e importante como Spam. Es el error más costoso, ya que oculta información vital para el usuario, derivando en problemas laborales o legales.
- **Falso Negativo (FN):** Permitir que un correo Spam llegue a la bandeja de entrada. Es un inconveniente menor, pues el usuario suele identificar la publicidad no deseada.
- **Métrica Elegida:** Dado que el costo de un FP es crítico y existe un ligero desbalance de clases, no se evalúa con *Accuracy*. Se priorizan el F1-Score, la Precisión y la curva PR-AUC, garantizando un estricto control del umbral de decisión para proteger los correos legítimos.

1.4. 1.4 Riesgos Éticos y Medidas Concretas

Riesgo: Violación de la privacidad y filtración de datos confidenciales contenidos en los correos evaluados.
Medidas Concretas: Procesamiento exclusivo en memoria volátil (RAM) sin almacenamiento ni registro en bases de datos (ver Figura 8); mecanismo transparente de aviso ético al usuario y derivación simulada a soporte humano (ver Figura 9).

2. Datos y Exploración sin Fuga (Etapa E2)

2.1. 2.1 Carga, Fuente y Estructura

Se trabajó con el dataset UCI Spambase (ID 94), conformado por 4,601 registros y 57 variables predictoras continuas más la clase binaria (1 = Spam, 0 = No Spam). Se generó un diccionario de variables y se registró el hash SHA-256 para trazabilidad.

2.2. 2.2 Auditoría de Calidad

- **Valores Faltantes:** 0 nulos nativos. Se mantuvo un *SimpleImputer* por mediana en el pipeline por robustez en producción.
- **Duplicados:** Se detectaron 391 filas duplicadas. Para evitar la contaminación (*Data Leakage*) entre particiones de entrenamiento y prueba, se mantuvieron y se aplicó particionamiento por huella / grupos.
- **Outliers:** Valores extremos en `char_freq_!` o `word_freq_free` se conservaron conscientemente, pues representan patrones reales de comportamiento publicitario y no errores de medición.

2.3. 2.3 Balance de Clases y Partición Estratificada

El dataset presenta 2,788 No Spam (60.60 %) y 1,813 Spam (39.40 %). Para evitar fuga de información (*Data Leakage*):

1. La partición se realizó antes de cualquier transformación: Entrenamiento (60 %, 2,760), Validación (20 %, 920) y Prueba (20 %, 921) con estratificación y semilla fija (`seed=42`).
2. El escalador (*StandardScaler*) y el imputador ajustaron sus parámetros (*fit_transform*) únicamente con `X_train`, aplicando solo `.transform()` sobre validación y prueba.

3. Modelado y Comparación (Etapa E3)

Se entrenaron modelos de complejidad creciente sobre `X_train_p` (ver evidencia de entrenamiento en la Figura 1):

- **Línea Base Trivial (DummyClassifier):** Predice la clase mayoritaria (No Spam). F1-Score en Validación = 0.0000.
- **Línea Base Simple (LogisticRegression):** Modelo lineal de referencia. F1-Score en Validación = 0.8380.
- **Red Neuronal MLP v1:** 1 capa oculta de 50 neuronas. F1-Score en Validación = 0.9183.
- **Red Neuronal MLP v2:** 2 capas ocultas (64, 32 neuronas), regularización L2 y parada temprana (*early stopping*). F1-Score en Validación = 0.9179.

Conclusión E3: La red neuronal supera de forma estadísticamente significativa a las líneas bases lineales al capturar interacciones no lineales entre frecuencias de palabras.

4. Evaluación y Versionado (Etapa E4)

4.1. 4.1 Elección y Aislamiento del Conjunto de Prueba

La selección del modelo ganador se basó exclusivamente en el rendimiento sobre el conjunto de Validación (ver ejecución en la Figura 2). El conjunto de Prueba (921 muestras) se mantuvo completamente aislado y fue evaluado una sola vez al final:

Modelo	F1 (Val)	F1 (Prueba)	Precisión	Recall
Dummy Baseline	0.0000	0.0000	0.0000	0.0000
Regresión Logística	0.8380	0.8819	0.9021	0.8625
MLP v1 (Ganador)	**0.9183**	**0.9224**	**0.9380**	**0.9074**
MLP v2	0.9179	0.9154	0.9315	0.9000

4.2. 4.2 Versionado y Trazabilidad

Se generaron fichas JSON por versión (`modelos/ficha_*.json`) registrando fecha, hiperparámetros y métricas. El modelo `mlp_v1` fue serializado y promovido como `modelo_produccion.joblib`.

5. Despliegue y API RESTful (Etapa E5)

Construida con FastAPI (`src/api.py`), exponiendo documentación interactiva automática (ver Figura 3):

- **Carga en Memoria (Warm-up):** Carga `modelo_produccion.joblib` y el preprocesador al arrancar el servidor para optimizar latencia.
- **Endpoints:** `GET /salud` (ver Figura 4) y `POST /predecir` (ver Figura 5).
- **Validación de Errores:** Esquemas Pydantic interceptan dimensiones inválidas, retornando HTTP 400 Bad Request (ver Figura 6) o HTTP 422.

6. Asistente Conversacional y Chatbot (Etapa E6)

Implementado en Streamlit (`src/chatbot.py`):

- **Flujo Multilingüe y Diagnóstico:** Permite ingresar correos en español; calcula las 57 características, consulta a la API y detalla alertas (ver Figura 7).
- **Explicabilidad y Ética:** Muestra aviso de privacidad en RAM (ver Figura 8).
- **Caminos del Chatbot:** Flujo completo documentado en el diagrama arquitectónico (ver Figura 10). Incluye manejo de fallbacks y derivación a soporte (ver Figura 9).

7. Garantía de Calidad y Reproducibilidad (Etapa E7)

- **Suite de Pruebas Automáticas (`pytest`):** 6 casos de prueba en verde (`PASSED`) cubriendo datos, API y chatbot (ver ejecución exitosa en la Figura 11).
- **Reproducibilidad:** Entorno congelado en `requirements.txt` / `pyproject.toml` y guía en `README.md`.

8. Conclusiones y Trabajo Futuro (Etapa E8)

El sistema cumple a cabalidad con la integración de Machine Learning, API y chatbot conversacional. Como trabajo futuro, se propone migrar la extracción heurística a un modelo de lenguaje (LLM/Transformer) adaptado directamente al español.

Aportes y Contribuciones Individuales

Integrante 1 – Planteamiento, Ética, Datos y Preprocesamiento (E1 y E2)
Emary Galarza Quilca: Contribuí a la definición del problema, identificando al usuario objetivo, el momento de predicción y el costo asimétrico de los errores (Falsos Positivos críticos). En la etapa de datos, realicé la auditoría de calidad, gestión de duplicados, inspección de *outliers* y diseñé la partición estricta sin fuga de datos usando estratificación y semilla fija.

Integrante 2 – Modelado y Entrenamiento (E3)
Miguel Angel Huiza Vasquez: Construí la línea base trivial y la línea base simple (Regresión Logística). Diseñé y entrené dos versiones de Redes Neuronales MLP aplicando regularización con *Dropout* y *Early Stopping*. Estructuré el script global de entrenamiento y evalué con franqueza el desempeño frente a los modelos lineales.

Integrante 3 – Evaluación, Versionado y API (E4 y E5)
Ares Gabriel Castillo Ayarza: Desarrollé el sistema de evaluación con métricas avanzadas (F1-Score, PR-AUC, matriz de confusión) y la selección del modelo ganador. Desplegué la API en FastAPI incorporando warm-up, esquemas Pydantic para las 57 características y manejo robusto de errores HTTP 400/422.

Integrante 4 – Chatbot Conversacional, Pruebas y Calidad (E6, E7 y E8)
Italo Osmar Grijalva Martinez: Implementé el chatbot en Streamlit con traducción multilingüe al vuelo, módulo heurístico, explicabilidad de alertas, manejo de *fallbacks* y privacidad en RAM. Diseñé el diagrama de flujo, estructuré la suite de pruebas en `pytest` y consolidé la reproducibilidad y documentación general del proyecto.

Declaración sobre el Uso de Inteligencia Artificial Generativa

En cumplimiento con los estándares de integridad académica de la asignatura, se declara que para el desarrollo del presente proyecto se contó con el apoyo asistido de herramientas de Inteligencia Artificial Generativa como asistente de co-pilotaje, refinamiento y optimización de código.

1. Marco de Asistencia y Ámbitos de Aplicación

- **Generación de Plantillas y Boilerplate:** Asistencia en la sintaxis inicial para esquemas Pydantic en FastAPI y configuración de pruebas unitarias con `pytest`.
- **Documentación y Diagramación:** Soporte en la redacción estilizada del informe y la síntesis en sintaxis Mermaid para el diagrama de arquitectura.
- **Traducción y Soporte Multilingüe:** Integración de librerías de traducción automática al vuelo en el chatbot.

2. Trabajo Humano, Análisis de Datos y Desarrollo Propio

Se hace especial énfasis en que el análisis exploratorio de datos, las decisiones de preprocesamiento y el desarrollo experimental contenido en el **Jupyter Notebook** (`notebooks/`) constituyeron un trabajo fundamentalmente técnico, manual e intelectual por parte del equipo:

- **Criterio Técnico de Limpieza:** La auditoría de los 4,601 registros y la decisión de conservar *outliers* derivaron de un análisis analítico de ciberseguridad propio del equipo.
- **Aislamiento de Fuga (*Data Leakage*):** El diseño de la partición estratificada y el ajuste del escalador exclusivamente sobre entrenamiento fueron verificados paso a paso por los integrantes.
- **Supervisión y Validación:** Toda recomendación de la IA fue rigurosamente auditada, probada y adaptada a la lógica del negocio y rúbrica del curso.

Anexos: Evidencias de Ejecución

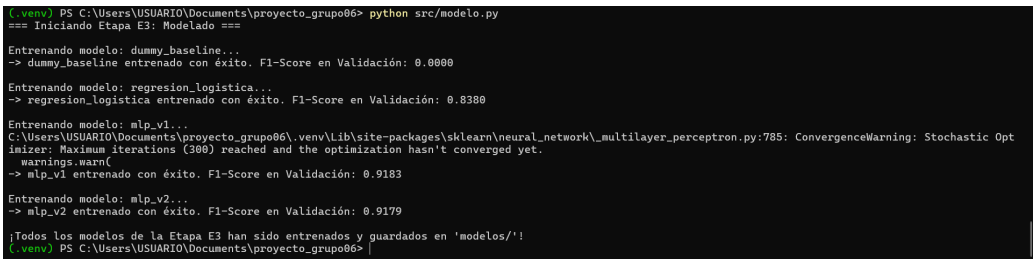


Figura 1: Entrenamiento de modelos de la Etapa E3 y evaluación de F1-Score en validación.

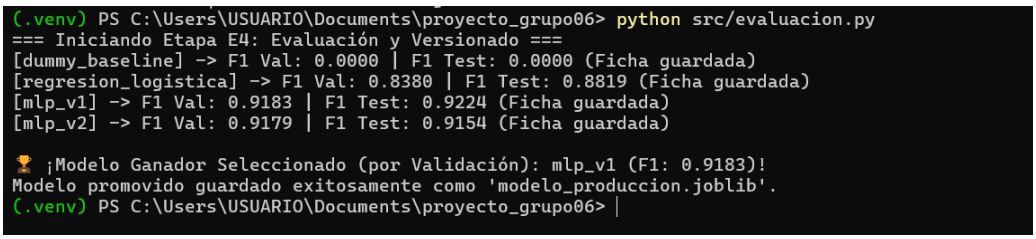


Figura 2: Selección del modelo ganador (MLP v1) y promoción a producción (`modelo_produccion.joblib`).



Figura 3: Documentación interactiva Swagger UI de la API RESTful (FastAPI).

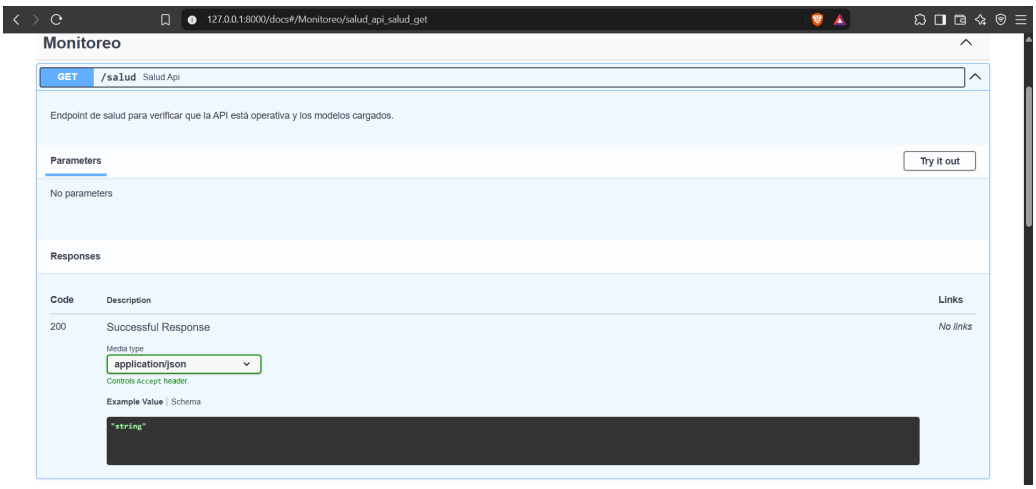


Figura 4: Verificación operativa de la API RESTful en la ruta `/salud` (HTTP 200 OK).

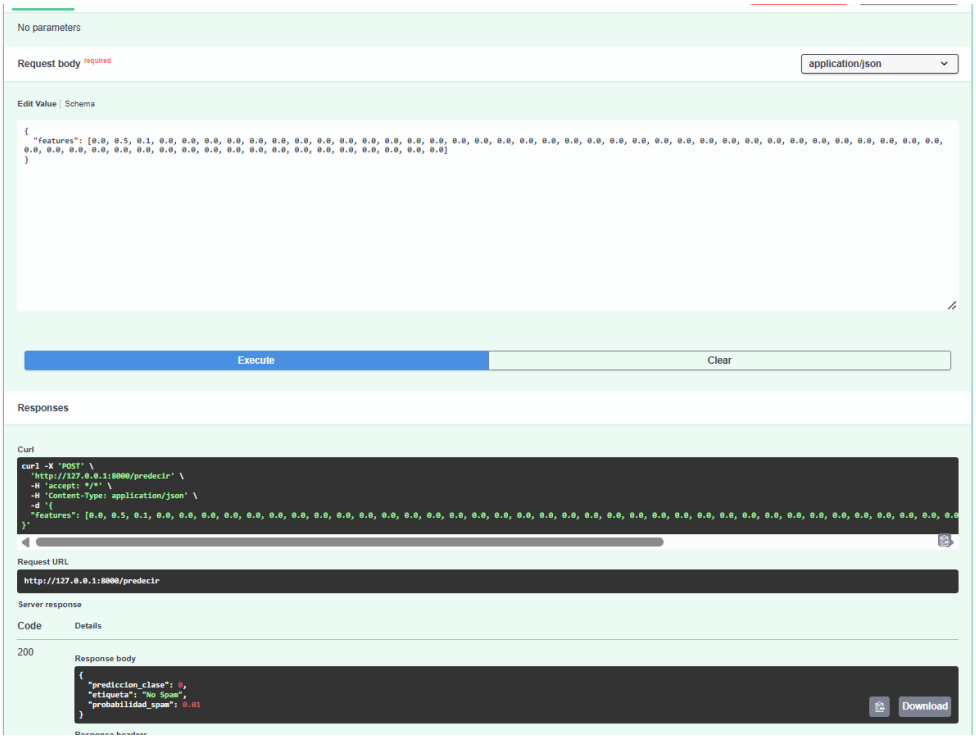


Figura 5: Prueba de inferencia exitosa en la API (`POST /predecir`) procesando el vector de 57 variables.

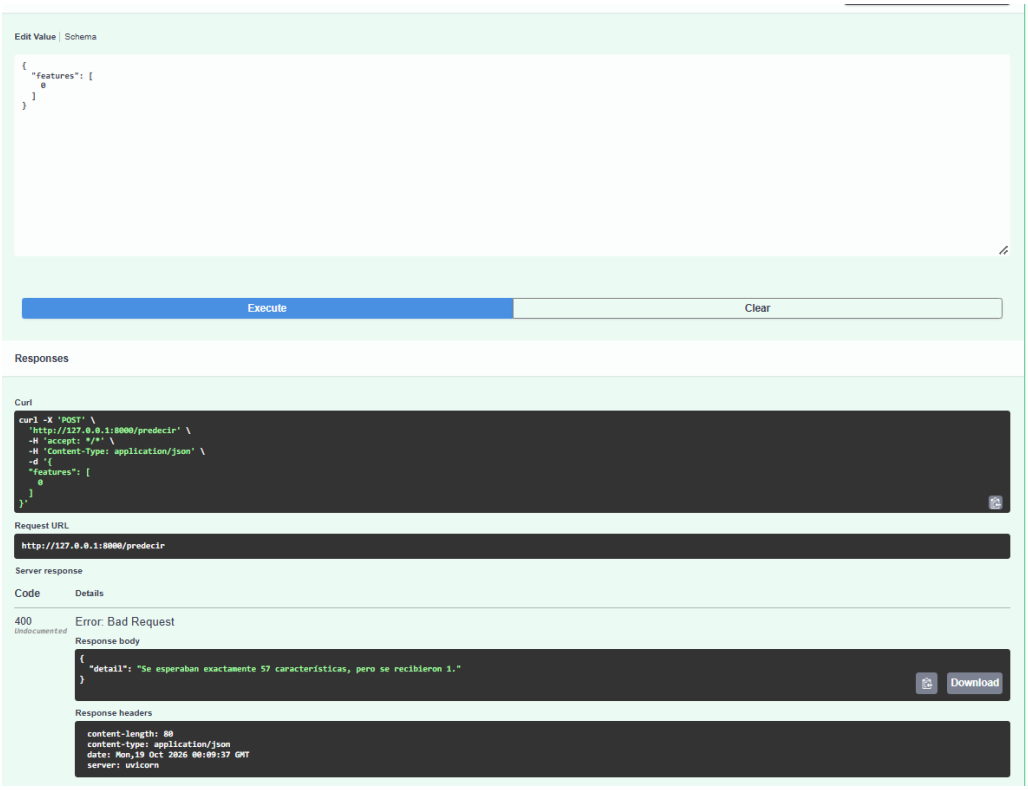


Figura 6: Manejo controlado de errores en la API devolviendo un código HTTP 400 Bad Request ante vectores con dimensiones incorrectas.

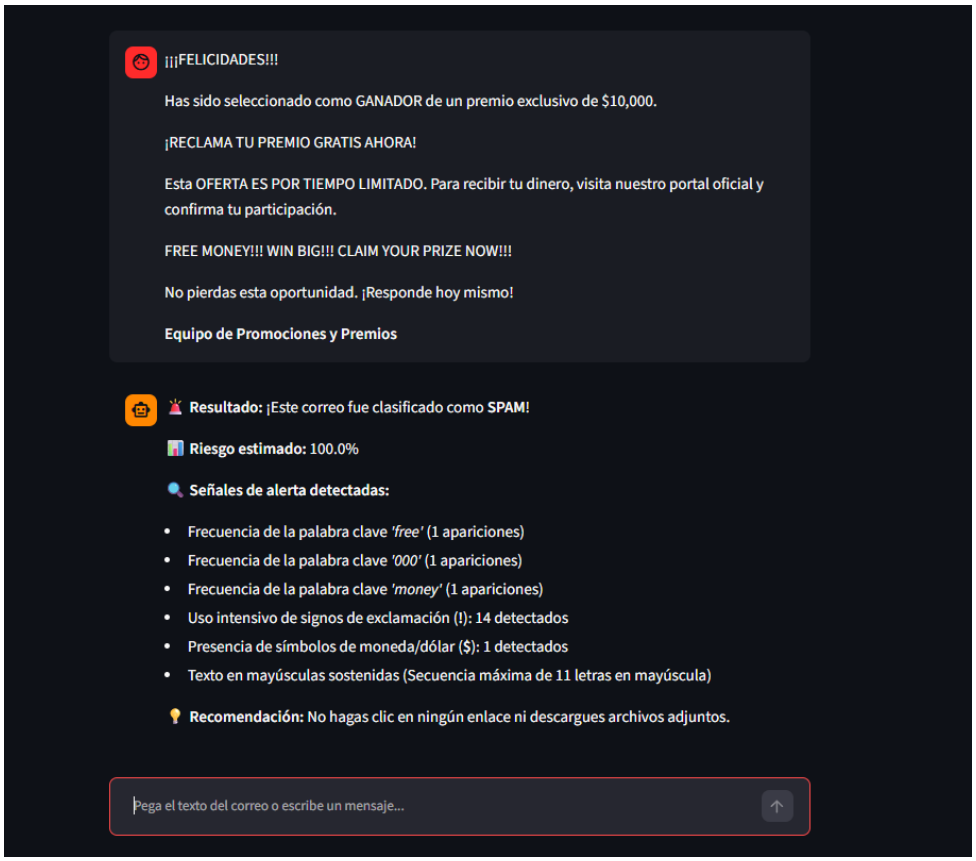


Figura 7: Interfaz del Chatbot en Streamlit mostrando la detección de Spam, porcentaje de riesgo, señales de alerta y recomendaciones.



Figura 8: Aviso de Privacidad y Cuidado Ético visible en la interfaz del chatbot (procesamiento en memoria RAM volátil).

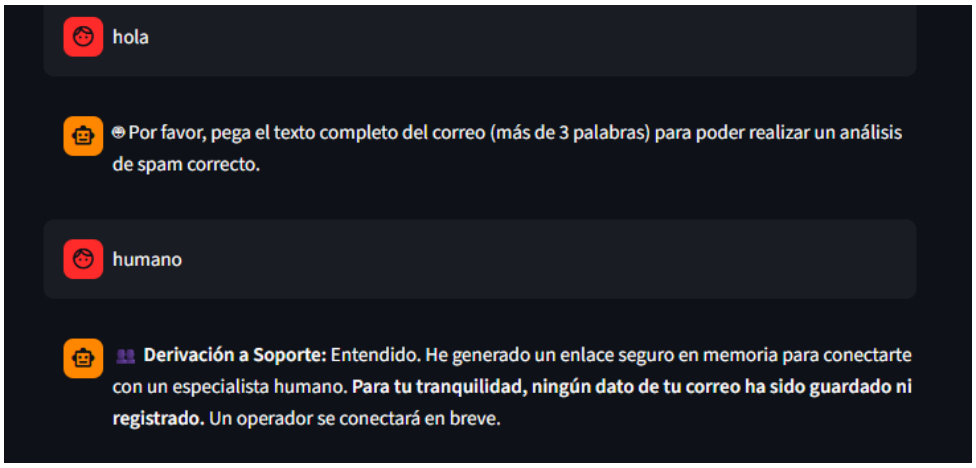


Figura 9: Respuesta de Fallback orientativo ante entradas cortas y simulación de derivación a soporte humano.

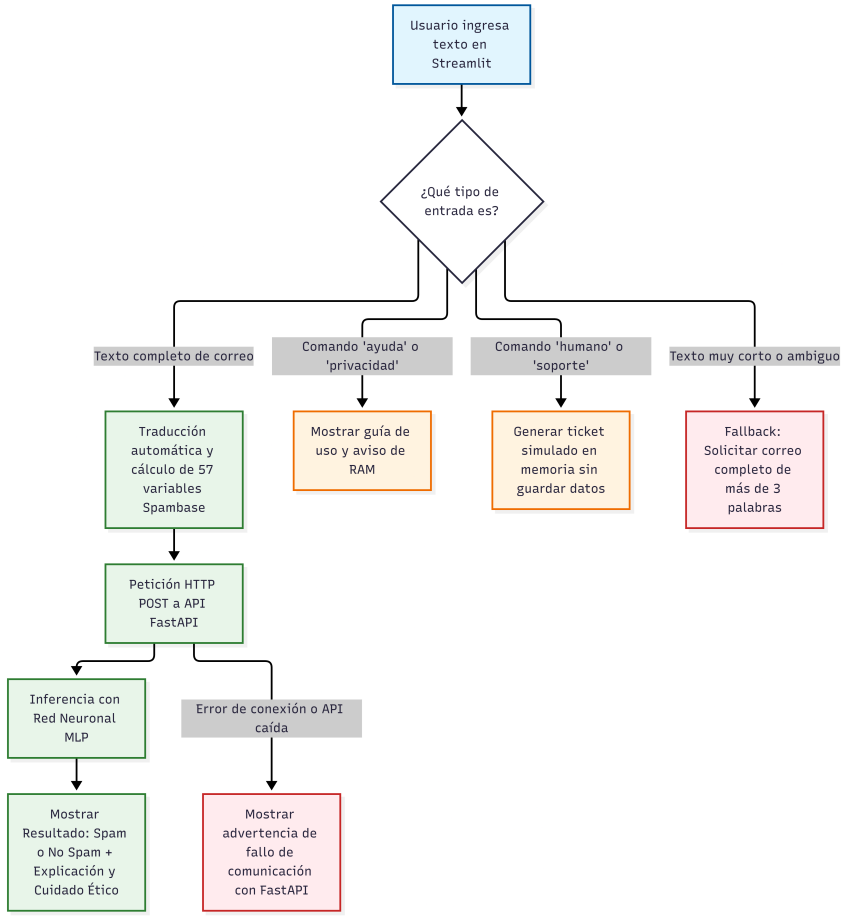


Figura 10: Diagrama de flujo del chatbot abarcando caminos feliz, alternativo y de error.

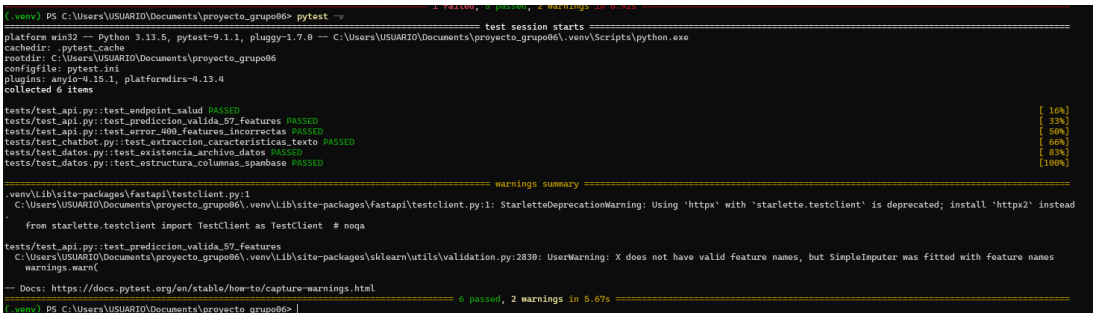


Figura 11: Ejecución exitosa de la suite de pruebas automáticas con `pytest` (100 % en verde).